UNIVERSITY OF ARKANSAS

CSCE 4133/5133 Algorithms

Homework Assignment #5

Submission Deadline: Mid-Night (11:59PM) May 1, 2026.

NO EXTENSION

Instructions

- **Submissions:** Students can use either Google Docs or LaTeX for writing the report. The report template is provided in the Blackboard (Material/Report-template.zip). Your submission should be a zip file containing your report and the source code, including your implementation. Name the zip file as `lastname_studentID.zip`. Submissions should be made via BlackBoard.
- **Policy:** Students must start working on this assignment independently and review the late-day policy.
- If you have any questions please contact us via Teams or email: kimtran@uark.edu and maalexra@uark.edu.

I. Overview

This assignment will help you practice and understand three fundamental algorithmic paradigms in Python:

- **String Algorithm (Rabin-Karp)**
- **Dynamic Programming (Floyd-Warshall all)**
- **NP-Completeness (Traveling Salesperson Problem)**

You need to work on only 1 file: **algorithms.py**

II. Implementation

II..1 String Algorithm: Rabin-Karp Algorithm

What is it?

A string-searching algorithm that uses hashing to find any one of a set of pattern strings in a text. It is particularly efficient for multiple pattern searches and is known for its use of a "rolling hash."

How does it work?

- Computes a hash value for the pattern and for every m-length substring of the text.
- If the hash values match, it performs a character-by-character comparison to handle potential hash collisions.
- Uses a rolling hash function to update the hash of the next substring in constant time $\mathcal{O}(1)$.
- Time complexity: Average $\mathcal{O}(n + m)$, Worst-case $\mathcal{O}(n \cdot m)$.

Function in the project:

- **Input:** A text and a pattern.
- **Output:** All starting indices where the pattern occurs in the text.
- **Use case:** Detecting plagiarism, identifying repeated DNA sequences, and searching for multiple patterns simultaneously.

II..2 Dynamic Programming: Floyd-Warshall Algorithm

What is it?

An all-pairs shortest path algorithm that finds the shortest distances between every pair of vertices in a weighted graph. It is a classic example of dynamic programming, breaking the problem down into subproblems based on intermediate vertices.

How does it work?

- Maintains a 2D matrix where $dist[i][j]$ represents the shortest distance from vertex i to vertex j.

- Iteratively updates the matrix by considering each vertex k as an intermediate point.
- The core transition: $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$.
- Time complexity: $\mathcal{O}(V^3)$, where V is the number of vertices.

Function in the project:

- **Input:** An adjacency matrix representing the graph.
- **Output:** A matrix containing the shortest distance between all pairs of buildings.
- **Use case:** Pre-calculating campus-wide distances to provide instant navigation results for any start and end point.

II..3 NP-Completeness: Traveling Salesperson Problem (TSP)

What is it?

A classic optimization problem that asks: "Given a list of cities and the distances between them, what is the shortest possible route that visits each city exactly once and returns to the origin city?" It is a well-known NP-hard problem.

How does it work?

- Uses Dynamic Programming with Bitmasking to reduce the complexity from $\mathcal{O}(n!)$ to $\mathcal{O}(2^n \cdot n^2)$.
- A "mask" is used to keep track of the set of visited cities using bits.
- The state is defined as $dp(mask, pos)$, representing the minimum cost to visit the remaining cities starting from the current position pos.

Function in the project:

- **Input:** A distance matrix and a set of locations to visit.
- **Output:** The minimum cost to complete a full tour of the designated locations.
- **Use case:** Planning an optimized campus tour that visits multiple interest points (e.g., student unions, labs, and libraries) with minimum walking.

The source file for algorithms.py should be as follows:

```python
def rabin_karp_search(pattern: str, text: str) -> list[int]:
    matches = []
    # YOUR CODE HERE
    return matches

def floyd_warshall(V: int, dist: list[list[int]]) -> None:
    # YOUR CODE HERE
    pass

```

```python
def solve_tsp(mask: int, pos: int, n: int, dist: list[list[int]], dp: list[list[int]]) -> int:
    # YOUR CODE HERE
    return -1  # Placeholder return value, replace with actual TSP solution
```

III. Compilation and Testing

Use Python3 to run, If you compile and run the sorce code, you should see the following:

```
=======================================================
   ALGORITHM 1: SMART CAMPUS DIRECTORY SEARCH
=======================================================
Action: Scanning campus database for: [HILL]
Method: Rabin-Karp Rolling Hash (Digital Fingerprinting)
-------------------------------------------------------
STATUS: Building not found.
Check spelling or update the campus catalogue.
```

If you implement all methods correctly, you should receive an output as follows:

```
=======================================================
   ALGORITHM 1: SMART CAMPUS DIRECTORY SEARCH
=======================================================
Action: Scanning campus database for: [HILL]
Method: Rabin-Karp Rolling Hash (Digital Fingerprinting)
-------------------------------------------------------
STATUS: Success! Building identified.
Location: Found 'HILL' at position 10 in the record.
Visual: BELL_JBHT_HILL_WJWH_HAPG
                  ^^^^
-------------------------------------------------------

=======================================================
   ALGORITHM 2: CAMPUS-WIDE NAVIGATION CALCULATOR
=======================================================
Status: Analyzing all possible walking routes...
Success: Shortest paths between all buildings calculated.

How to read the table below:
- The numbers represent the minimum number of 'steps' between buildings.
- 'INF' means there is currently no connected path between those two buildings.
-------------------------------------------------------
FROM \ TO       BELL    JBHT    HILL    WJWH    HAPG
-----------------------------------------------------------
BELL |          0       2       1       2       3
JBHT |          2       0       1       2       3
HILL |          1       1       0       1       2
WJWH |          2       2       1       0       1
HAPG |          3       3       2       1       0
-------------------------------------------------------
Example: To get from BELL to JBHT, it takes 2 connection(s).

=======================================================
   ALGORITHM 3: EFFICIENT CAMPUS TOUR PLANNER (TSP)
```

```
=======================================================
Mission: Visit 4 locations and return to start with minimum walking.
Target Stops: BELL -> JBHT -> HILL -> WJWH -> [Return]
Status: Solving the 'Traveling Salesperson' puzzle...
-------------------------------------------------------
RESULT: Optimal route identified.
Total Walking Distance: 6 connections.
Efficiency: This is the shortest possible path out of all 4! potential combinations.
=======================================================
```

IV. Submission

Write a report detailing the key ideas for implementing each required function and include the implemented functions themselves. You must also include a screenshot of the final output (from either command prompt or terminal) after running the program.

Write your full name, email address and student ID in the report. Your submission should be a zip file of your report and the source code, including your implementation. Name the zip file as 'lastname_studentID.zip'. Submission via BlackBoard.